

LeetCode 55

Jump Game

© Authored by Arpit Raj, LNMIIT Jaipur | August 8, 2026

Problem Understanding

You are given an array:

```
nums = [2, 3, 1, 1, 4]
```

You start at index 0.

`nums[i]` tells you the maximum number of positions you can jump forward from index `i`.

For example:

index:	0	1	2	3	4
nums:	2	3	1	1	4

From index 0, you can go:

0 → 1

or

0 → 2

From index 1, you can jump up to 3 positions.

Approach 1: Dynamic Programming (Memoization / Tabulation)

```
class Solution {
public:
    bool canJump(vector<int>& nums) {

        int n=nums.size();

        // dp[idx] tell if idx is reachable or not
        vector<int> dp(n,0);
        dp[n-1]=1; //last node pe khade hue hokar last node pe to phuch hi skte

        //peeche se aage traverse krenge
        for(int i=n-2;i>=0;i--){
            for(int j=1;j<=nums[i];j++){

                //ek ek krke saare steps leke dekhenge
                if(i+j<n && dp[i+j]){
                    dp[i]=1;
                }
            }
        }

        if(dp[0]==1) return true;
        else return false;
    }
};
```

Approach 2: Greedy (Optimized)

```
class Solution {
public:
    bool canJump(vector<int>& nums) {
        int n=nums.size();
        int farthest=0;

        for(int i=0;i<n;i++){
            if(i>farthest) return false;
            // current hi max potential se door he isliye

            farthest=max(farthest,i+nums[i]);
            if(farthest>=n-1){
                return true;
            }
        }
        return false;
    }
};
```